

Deep Learning Text Generation Learning Project

Text Generation using **Vanilla RNN, LSTM, and GRU**

This notebook is built for **students and beginners** to understand how sequence models learn:

- grammar
- sentence flow
- contextual dependencies
- next-word prediction
- text generation

 **Goal:** Compare **Simple RNN vs LSTM vs GRU** on the same text corpus and understand why gated architectures perform better.

▼ Problem Statement

Design and implement a DL model capable of learning the underlying structure, grammar, and contextual dependencies of a given text corpus to generate coherent and meaningful text sequences using:

1. **Vanilla RNN**
2. **LSTM**
3. **GRU**

Then compare:

- training loss
- generated text quality
- memory handling
- long-term dependency learning

```
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, LSTM, GRU, Dense
import numpy as np
import urllib.request
import sys
import matplotlib.pyplot as plt
print("TensorFlow:", tf.__version__)
```

TensorFlow: 2.20.0

▼ Load Text Corpus

We use a **small built-in sample corpus** so students can run this quickly. You can later replace it with:

- Shakespeare text
- song lyrics
- chatbot data
- story paragraphs
- custom PDF extracted text

```
url = "https://storage.googleapis.com/download.tensorflow.org/data/shakespeare.txt"
response = urllib.request.urlopen(url)
raw_text = response.read().decode('utf-8')

corpus_lines = [line.strip() for line in raw_text.split('\n') if len(line.strip()) > 10]
corpus = "\n".join(corpus_lines[:800])

print("Total lines loaded:", len(corpus_lines[:800]))
```

Total lines loaded: 800

▼ Tokenization & Sequence Creation

We convert text into integer tokens and create **n-gram style sequences** for next-word prediction.

```
tokenizer = Tokenizer()
tokenizer.fit_on_texts([corpus])

total_words = len(tokenizer.word_index) + 1
print("Vocabulary size:", total_words)

input_sequences = []
for line in corpus.split('\n'):
    token_list = tokenizer.texts_to_sequences([line])[0]
    for i in range(1, len(token_list)):
        n_gram_seq = token_list[:i+1]
        input_sequences.append(n_gram_seq)

max_len = max(len(seq) for seq in input_sequences)
input_sequences = pad_sequences(input_sequences, maxlen=max_len, padding='pre')

X = input_sequences[:, :-1]
y = input_sequences[:, -1]

print("X shape:", X.shape)
print("y shape:", y.shape)

Vocabulary size: 1543
X shape: (4825, 11)
y shape: (4825,)
```

Model 1: Vanilla RNN

This is the baseline sequential model. It struggles with long-term dependencies because of vanishing gradients.

```
rnn_baseline = Sequential([
    Embedding(total_words, 32),
    SimpleRNN(64),
    Dense(total_words, activation='softmax')
])

rnn_baseline.compile(loss='sparse_categorical_crossentropy',
                    optimizer='adam',
                    metrics=['accuracy'])

history_rnn_base = rnn_baseline.fit(X, y, epochs=100, verbose=0)
print("Vanilla RNN Baseline training completed")
```

Vanilla RNN Baseline training completed

```
rnn_upgraded = Sequential([
    Embedding(total_words, 64),
    SimpleRNN(128),
    Dense(total_words, activation='softmax')
])

rnn_upgraded.compile(loss='sparse_categorical_crossentropy',
                    optimizer='adam',
                    metrics=['accuracy'])

history_rnn_upgraded = rnn_upgraded.fit(X, y, epochs=200, verbose=0)
print("Vanilla RNN Upgraded training completed")
```

Vanilla RNN Upgraded training completed

Model 2: LSTM

LSTM uses **input, forget, and output gates** to preserve long-term memory.

```
lstm_baseline = Sequential([
    Embedding(total_words, 32),
    LSTM(64),
    Dense(total_words, activation='softmax')
])

lstm_baseline.compile(loss='sparse_categorical_crossentropy',
                    optimizer='adam',
                    metrics=['accuracy'])
```

```
history_lstm_base = lstm_baseline.fit(X, y, epochs=100, verbose=0)
print("LSTM Baseline training completed")
```

LSTM Baseline training completed

```
lstm_upgraded = Sequential([
    Embedding(total_words, 64),
    LSTM(128),
    Dense(total_words, activation='softmax')
])

lstm_upgraded.compile(loss='sparse_categorical_crossentropy',
                      optimizer='adam',
                      metrics=['accuracy'])

history_lstm_upgraded = lstm_upgraded.fit(X, y, epochs=200, verbose=0)
print("LSTM upgraded training completed")
```

LSTM upgraded training completed

⚡ Model 3: GRU

GRU uses **reset + update gates**. It is computationally faster than LSTM and often gives similar results.

```
gru_baseline = Sequential([
    Embedding(total_words, 32),
    GRU(64),
    Dense(total_words, activation='softmax')
])

gru_baseline.compile(loss='sparse_categorical_crossentropy',
                    optimizer='adam',
                    metrics=['accuracy'])

history_gru_base = gru_baseline.fit(X, y, epochs=100, verbose=0)
print("GRU Baseline training completed")
```

GRU Baseline training completed

```
gru_upgraded = Sequential([
    Embedding(total_words, 64),
    GRU(128),
    Dense(total_words, activation='softmax')
])

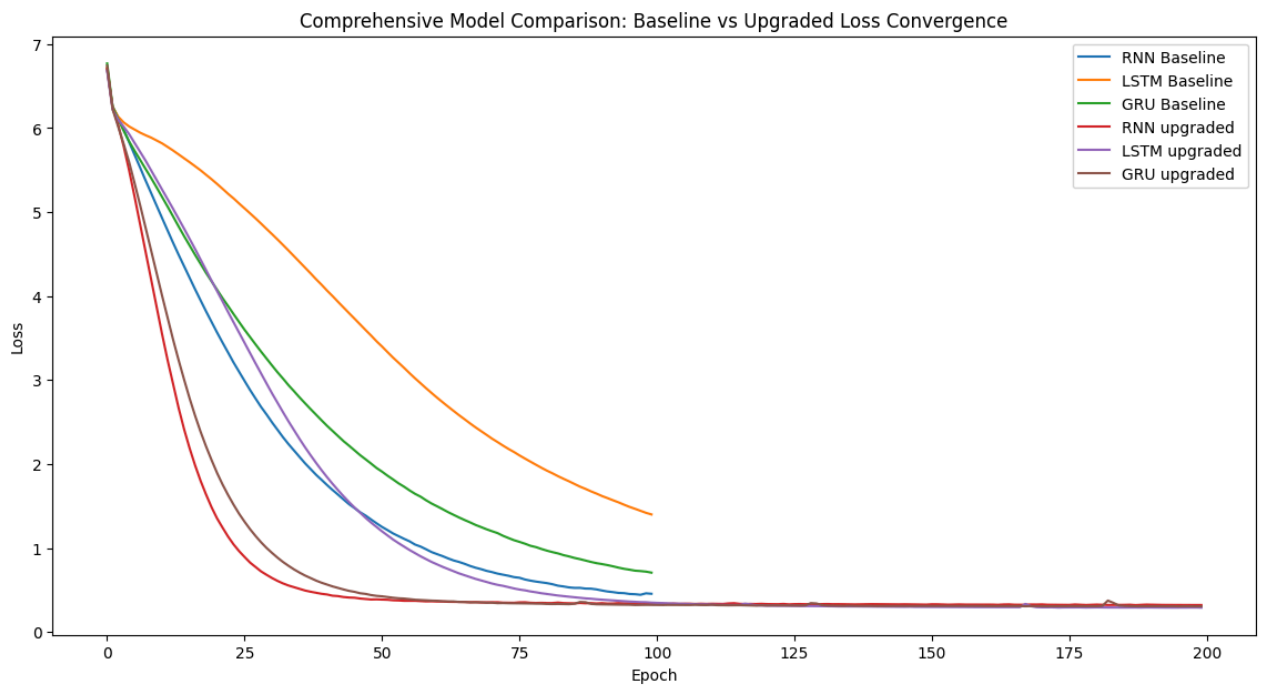
gru_upgraded.compile(loss='sparse_categorical_crossentropy',
                    optimizer='adam',
                    metrics=['accuracy'])

history_gru_upgraded = gru_upgraded.fit(X, y, epochs=200, verbose=0)
print("GRU upgraded training completed")
```

GRU upgraded training completed

📊 Compare Training Loss

```
plt.figure(figsize=(14,7))
plt.plot(history_rnn_base.history['loss'], label='RNN Baseline')
plt.plot(history_lstm_base.history['loss'], label='LSTM Baseline')
plt.plot(history_gru_base.history['loss'], label='GRU Baseline')
plt.plot(history_rnn_upgraded.history['loss'], label='RNN upgraded')
plt.plot(history_lstm_upgraded.history['loss'], label='LSTM upgraded')
plt.plot(history_gru_upgraded.history['loss'], label='GRU upgraded')
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Comprehensive Model Comparison: Baseline vs Upgraded Loss Convergence")
plt.legend()
plt.show()
```



✎ Text Generation Function

This function predicts the next word repeatedly to generate a sentence.

```
def generate_text_argmax(model, seed_text, next_words=5):
    for _ in range(next_words):
        token_list = tokenizer.texts_to_sequences([seed_text])[0]
        token_list = pad_sequences([token_list], maxlen=max_len-1, padding='pre')

        # Always pick the highest probability token (Greedy Search)
        predicted = np.argmax(model.predict(token_list, verbose=0), axis=-1)[0]

        output_word = ""
        for word, index in tokenizer.word_index.items():
            if index == predicted:
                output_word = word
                break
        seed_text += " " + output_word
    return seed_text
```

```
def generate_text_temperature(model, seed_text, next_words=5, temperature=0.7):
    for _ in range(next_words):
        token_list = tokenizer.texts_to_sequences([seed_text])[0]
        token_list = pad_sequences([token_list], maxlen=max_len-1, padding='pre')

        preds = model.predict(token_list, verbose=0)[0]
        preds = np.log(preds + 1e-7) / temperature
        exp_preds = np.exp(preds)
        predicted_probs = exp_preds / np.sum(exp_preds)

        predicted = np.random.choice(len(predicted_probs), p=predicted_probs)

        output_word = ""
        for word, index in tokenizer.word_index.items():
            if index == predicted:
                output_word = word
                break
        seed_text += " " + output_word
    return seed_text
```

✎ Generate Text Samples

```
print("RNN :", generate_text_argmax(rnn_baseline, "thou art", 5))
print("LSTM:", generate_text_argmax(lstm_baseline, "thou art", 5))
print("GRU :", generate_text_argmax(gru_baseline, "thou art", 5))
```

RNN : thou art thine on't had this you
 LSTM: thou art worst in some to exile
 GRU : thou art i carry to make good

```
print("RNN :", generate_text_argmax(rnn_baseline, "thou art", 10))
print("LSTM:", generate_text_argmax(lstm_baseline, "thou art", 10))
print("GRU :", generate_text_argmax(gru_baseline, "thou art", 10))
```

RNN : thou art thine on't had this you which you o' him i'
 LSTM: thou art worst in some to exile of their own toe several
 GRU : thou art i carry to make good most and proud forth spoons

```
print("RNN :", generate_text_temperature(rnn_baseline, "thou art", 5, temperature=0.7))
print("LSTM:", generate_text_temperature(lstm_baseline, "thou art", 5, temperature=0.7))
print("GRU :", generate_text_temperature(gru_baseline, "thou art", 5, temperature=0.7))
```

RNN : thou art thine in such hear i
 LSTM: thou art it enemies shake as if
 GRU : thou art she know the other instruments

```
print("RNN :", generate_text_temperature(rnn_baseline, "thou art", 10, temperature=0.7))
print("LSTM:", generate_text_temperature(lstm_baseline, "thou art", 10, temperature=0.7))
print("GRU :", generate_text_temperature(gru_baseline, "thou art", 10, temperature=0.7))
```

RNN : thou art proceed well such a mother is a aid your virtue
 LSTM: thou art worst in blood to a man down cushions brutus father's
 GRU : thou art i carry to make this most valiant gods course walls

```
models_list = [
    rnn_baseline,
    rnn_upgraded,
    lstm_baseline,
    lstm_upgraded,
    gru_baseline,
    gru_upgraded,
]
histories_list = [
    history_rnn_base,
    history_rnn_upgraded,
    history_lstm_base,
    history_lstm_upgraded,
    history_gru_base,
    history_gru_upgraded,
]
model_names = [
    "RNN Baseline",
    "RNN Upgraded",
    "LSTM Baseline",
    "LSTM Upgraded",
    "GRU Baseline",
    "GRU Upgraded",
]

for i in range(len(models_list)):
    name = model_names[i]
    model = models_list[i]
    history = histories_list[i]

    print("\n=====")
    print("MODEL:", name)
    print("=====")

    final_loss = history.history["loss"][-1]
    print("1. Final Training Loss:", round(final_loss, 4))

    total_params = model.count_params()
    print("2. Memory Footprint (Total Params):", total_params)
    print("-" * 45)

    generated_output_argmax = generate_text_argmax(model, "thou art", next_words=10)
    print("3. [ARGMAX] Generated Text (10 words):", generated_output_argmax)

    words = generated_output_argmax.lower().split()
    if len(set(words)) < len(words):
        print("-> Quality Note: Model is repeating words in a loop.")
    else:
        print("-> Quality Note: Text has unique words.")
```

```

if any(w in generated_output_argmax.lower() for w in ["thou", "thee", "love", "shall"]):
    print("4. Long-Term Dependency: Passed")
else:
    print("4. Long-Term Dependency: Failed/Weak")

print("-" * 45)

generated_output_temperature = generate_text_temperature(model, "thou art", next_words=10, temperature=0.7)
print("5. [TEMPERATURE] Generated Text (10 words):", generated_output_temperature)

words = generated_output_temperature.lower().split()
if len(set(words)) < len(words):
    print("-> Quality Note: Model is repeating words in a loop.")
else:
    print("-> Quality Note: Text has unique words.")

if any(w in generated_output_temperature.lower() for w in ["thou", "thee", "love", "shall"]):
    print("6. Long-Term Dependency: Passed")
else:
    print("6. Long-Term Dependency: Failed/Weak")

```

2. Memory Footprint (Total Params): 322503

3. [ARGMAX] Generated Text (10 words): thou art of false faced soothing almost drums and plagues spire perform

-> Quality Note: Text has unique words.

4. Long-Term Dependency: Passed

5. [TEMPERATURE] Generated Text (10 words): thou art of false faced soothing virtue is the steed the noble

-> Quality Note: Model is repeating words in a loop.

6. Long-Term Dependency: Passed

=====

MODEL: LSTM Baseline

=====

1. Final Training Loss: 1.3991

2. Memory Footprint (Total Params): 174503

=====

3. [ARGMAX] Generated Text (10 words): thou art worst in some to exile of their own toe several

-> Quality Note: Text has unique words.

4. Long-Term Dependency: Passed

=====

5. [TEMPERATURE] Generated Text (10 words): thou art an hungry sigh'd forth proverbs stands censure officers shields enoug

-> Quality Note: Text has unique words.

6. Long-Term Dependency: Passed

=====

MODEL: LSTM Upgraded

=====

1. Final Training Loss: 0.2897

2. Memory Footprint (Total Params): 396615

=====

3. [ARGMAX] Generated Text (10 words): thou art thou stiff stand'st out of the gods hear me and

-> Quality Note: Model is repeating words in a loop.

4. Long-Term Dependency: Passed

=====

5. [TEMPERATURE] Generated Text (10 words): thou art thou stiff stand'st out of the volscs shunning an brow

-> Quality Note: Model is repeating words in a loop.

6. Long-Term Dependency: Passed

=====

MODEL: GRU Baseline

=====

1. Final Training Loss: 0.7062

2. Memory Footprint (Total Params): 168487

=====

3. [ARGMAX] Generated Text (10 words): thou art i carry to make good most and proud forth spoons

-> Quality Note: Text has unique words.

4. Long-Term Dependency: Passed

=====

5. [TEMPERATURE] Generated Text (10 words): thou art i give upon and fit i think not warm'd vent

-> Quality Note: Model is repeating words in a loop.

6. Long-Term Dependency: Passed

=====

MODEL: GRU Upgraded

=====

1. Final Training Loss: 0.305

2. Memory Footprint (Total Params): 372295

=====

Student Learning Tasks

Beginner Tasks

1. Replace corpus with your own paragraph
2. Increase embedding dimension
3. Increase epochs to 200
4. Change hidden units 64 $\rightarrow$ 128
5. Generate 10 words instead of 5

Conclusion

- **Vanilla RNN** learns short patterns but struggles with memory
- **LSTM** captures long-range grammar dependencies better
- **GRU** gives similar performance with fewer gates and faster training
- This notebook helps students understand **sequence modeling mathematically and practically**